

Claude Code + Lux: A Z Specification

Layer 2 — Visual Output Surface State

March 2026

Contents

1	Introduction	2
2	Base Types (from <code>claude-code.tex</code>)	2
3	Lux Types	3
4	Global Constants	3
5	State	3
6	Initialization	4
7	Lux Frame Condition	5
8	Session Lifecycle Operations	5
8.1	StartSession	5
8.2	SubmitPrompt	6
9	Tool Call Operations	6
9.1	BeginToolCall	6
9.2	PreToolHookAllow	7
9.3	PreToolHookDeny	7
9.4	GrantPermission	8
9.5	DenyPermission	8
9.6	CompleteToolSuccess	8
9.7	CompleteToolFailure	9
10	Lux Display Operations	9
10.1	EnableDisplay	9
10.2	DisableDisplay	10
10.3	ShowScene	10
10.4	UpdateScene	11
10.5	ClearDisplay	11
10.6	WaitForInteraction	12
10.7	ReceiveInteraction	12
11	Consumer Integration Operations	13
11.1	MarkBeadsBoardStale	13
12	Subagent Operations	13
13	Response, Stop, Context, and Session End	14
14	System Invariants	16

15 Consumer Integration Patterns	16
15.1 L0: Presence Detection	16
15.2 Beads Integration	17
15.3 Quarry Integration	17
15.4 PR Workflow Integration	17
15.5 Z-Spec Integration	17
15.6 Session Dashboard	17
16 Hook Event Map	17
17 Implementation Validation	18
17.1 Built and Correct	18
17.2 Not Yet Built	18
17.3 Design Note: Building Block Boundary	18

1 Introduction

Document status. This is a formal integration specification, not the canonical rewrite-target document. For the rewrite target, start with `target/target.md`. For the current/intermediate whole-system narrative, see `system.tex`.

This specification extends the base Claude Code state machine (`claude-code.tex`) with lux visual output surface state. Lux is a *building block* (see `integration.md`): it provides deterministic rendering at L1 that agents compose at L4. Lux has no upstream awareness — it does not know about beads, biff, quarry, or any consumer. Consumers discover lux at L1 and use it to render visual output.

The model captures two concerns:

1. **Lux display state** — whether the display is active, what scene is shown, whether an interaction is pending.
2. **Consumer integration** — how other tools trigger lux displays via hooks and skills, gated by a display mode flag.

The key properties verified:

1. **Scene requires display:** a scene can only be active when the display is enabled.
2. **No orphan interactions:** blocking for user interaction (`recv`) requires an active scene.
3. **Display mode gates rendering:** when `displayMode = dmOff`, no consumer triggers lux displays.
4. **Scene updates are safe:** `UpdateScene` requires an active scene. No updates to a cleared display.
5. **Beads board refresh:** after bead state changes (`ClaimBead`, `CloseBead`), the beads board is marked stale for refresh.

2 Base Types (from `claude-code.tex`)

$[TOOLNAME, TOOLINPUT, TOOLOUTPUT, PROMPT, RESPONSE, AGENTID, SESSIONID, CONTEXTCH$

$SessionPhase ::= spInactive \mid spIdle \mid spProcessing \mid spResponding \mid spEnded$

$ToolPhase ::= tpNone \mid tpPreHook \mid tpPermission \mid tpExecuting \mid tpPostHook$

$PermissionMode ::= pmDefault \mid pmPlan \mid pmAcceptEdits \mid pmDontAsk \mid pmBypass$

$PermissionDecision ::= pdAllow \mid pdDeny \mid pdAsk$

$HookDecision ::= hdAllow \mid hdBlock$

$SessionStartSource ::= ssStartup \mid ssResume \mid ssClear \mid ssCompact$

$SessionEndReason ::= seClear \mid seLogout \mid sePromptExit \mid seBypassDisabled \mid seOther$

$ZBOOL ::= ztrue \mid zfalse$

3 Lux Types

$[SCENEID, ELEMENTID]$

SCENEID identifies a displayed scene (e.g., `beads-board`, `quarry-search`, `z-spec-states`). ELEMENTID identifies an interactive element within a scene (e.g., a button, slider, or table row).

$DisplayMode ::= dmOff \mid dmOn$

Whether lux visual output is enabled. `dmOn` means consumers should render visual output when appropriate. `dmOff` means consumers degrade to text-only. Analogous to vox’s `notifyMode`.

$SceneKind ::= skBeads \mid skDashboard \mid skDataExplorer \mid skDiagram \mid skCustom$

Classification of the active scene. Used by consumer hooks to decide whether to update or replace the display.

4 Global Constants

$maxTools : \mathbb{N}$	$maxTools : \mathbb{N}$
$maxSubagents : \mathbb{N}$	$maxSubagents : \mathbb{N}$
$maxContext : \mathbb{N}$	$maxContext : \mathbb{N}$
$maxTools = 5$	$maxTools = 5$
$maxSubagents = 2$	$maxSubagents = 2$
$maxContext = 3$	$maxContext = 3$

5 State

State

$sessionId : \mathbb{F} \text{ SESSIONID}$
 $sessionPhase : \text{SessionPhase}$
 $permMode : \text{PermissionMode}$
 $toolPhase : \text{ToolPhase}$
 $currentTool : \mathbb{F} \text{ TOOLNAME}$
 $toolsThisTurn : \mathbb{N}$
 $activeSubagents : \mathbb{F} \text{ AGENTID}$
 $context : \text{seq } \text{CONTEXTCHUNK}$
 $pendingPrompt : \mathbb{F} \text{ PROMPT}$
 $pendingResponse : \mathbb{F} \text{ RESPONSE}$
 $displayMode : \text{DisplayMode}$
 $activeScene : \mathbb{F} \text{ SCENEID}$
 $sceneKind : \mathbb{F} \text{ SceneKind}$
 $pendingRecv : \text{ZBOOL}$
 $beadsBoardStale : \text{ZBOOL}$

$\#sessionId \leq 1$
 $sessionPhase = spInactive \Rightarrow sessionId = \emptyset$
 $sessionPhase \neq spInactive \Rightarrow \#sessionId = 1$
 $sessionPhase \neq spProcessing \Rightarrow toolPhase = tpNone$
 $\#currentTool \leq 1$
 $toolPhase = tpNone \Rightarrow currentTool = \emptyset$
 $toolPhase \neq tpNone \Rightarrow \#currentTool = 1$
 $toolsThisTurn \leq \text{maxTools}$
 $sessionPhase = spInactive \Rightarrow activeSubagents = \emptyset$
 $sessionPhase = spEnded \Rightarrow activeSubagents = \emptyset$
 $\#activeSubagents \leq \text{maxSubagents}$
 $\#context \leq \text{maxContext}$
 $\#pendingPrompt \leq 1$
 $sessionPhase = spIdle \Rightarrow pendingPrompt = \emptyset$
 $sessionPhase = spInactive \Rightarrow pendingPrompt = \emptyset$
 $sessionPhase = spEnded \Rightarrow pendingPrompt = \emptyset$
 $\#pendingResponse \leq 1$
 $sessionPhase \neq spResponding \Rightarrow pendingResponse = \emptyset$
 $\#activeScene \leq 1$
 $\#sceneKind \leq 1$
 $displayMode = dmOff \Rightarrow activeScene = \emptyset$
 $activeScene = \emptyset \Rightarrow sceneKind = \emptyset$
 $activeScene \neq \emptyset \Rightarrow \#sceneKind = 1$
 $pendingRecv = ztrue \Rightarrow activeScene \neq \emptyset$
 $beadsBoardStale = ztrue \Rightarrow sceneKind = \{skBeads\}$
 $sessionPhase = spInactive \Rightarrow displayMode = dmOff$
 $sessionPhase = spInactive \Rightarrow pendingRecv = zfalse$
 $sessionPhase = spInactive \Rightarrow beadsBoardStale = zfalse$

6 Initialization

Init
State'

sessionId' = $\emptyset$
sessionPhase' = *spInactive*
permMode' = *pmDefault*
toolPhase' = *tpNone*
currentTool' = $\emptyset$
toolsThisTurn' = 0
activeSubagents' = $\emptyset$
context' = $\langle \rangle$
pendingPrompt' = $\emptyset$
pendingResponse' = $\emptyset$
displayMode' = *dmOff*
activeScene' = $\emptyset$
sceneKind' = $\emptyset$
pendingRecv' = *zfalse*
beadsBoardStale' = *zfalse*

7 Lux Frame Condition

LuxFrame
 $\Delta State$

displayMode' = *displayMode*
activeScene' = *activeScene*
sceneKind' = *sceneKind*
pendingRecv' = *pendingRecv*
beadsBoardStale' = *beadsBoardStale*

8 Session Lifecycle Operations

8.1 StartSession

Lux reads display mode from config (*.lux/config.md* or equivalent sentinel). If no config exists, defaults to *dmOff*.

StartSession

Δ State

sid? : *SESSIONID*

source? : *SessionStartSource*

mode? : *PermissionMode*

initDisplay? : *DisplayMode*

sessionPhase = *spInactive*

sessionPhase' = *spIdle*

sessionId' = {*sid?*}

permMode' = *mode?*

toolPhase' = *tpNone*

currentTool' = $\emptyset$

toolsThisTurn' = 0

activeSubagents' = $\emptyset$

context' = $\langle \rangle$

pendingPrompt' = $\emptyset$

pendingResponse' = $\emptyset$

displayMode' = *initDisplay?*

activeScene' = $\emptyset$

sceneKind' = $\emptyset$

pendingRecv' = *zfalse*

beadsBoardStale' = *zfalse*

8.2 SubmitPrompt

SubmitPrompt

LuxFrame

prompt? : *PROMPT*

hookResult? : *HookDecision*

contextChunk? : *CONTEXTCHUNK*

sessionPhase = *spIdle*

hookResult? = *hdAllow*

sessionPhase' = *spProcessing*

pendingPrompt' = {*prompt?*}

context' = $(1 \dots \maxContext) \triangleleft (context \frown \langle contextChunk? \rangle)$

toolsThisTurn' = 0

toolPhase' = *tpNone*

currentTool' = $\emptyset$

sessionId' = *sessionId*

permMode' = *permMode*

activeSubagents' = *activeSubagents*

pendingResponse' = $\emptyset$

RejectPrompt

Ξ State

prompt? : *PROMPT*

hookResult? : *HookDecision*

sessionPhase = *spIdle*

hookResult? = *hdBlock*

9 Tool Call Operations

9.1 BeginToolCall

BeginToolCall

LuxFrame

tool? : *TOOLNAME*

sessionPhase = *spProcessing*
toolPhase = *tpNone*
toolsThisTurn < *maxTools*
toolPhase' = *tpPreHook*
currentTool' = {*tool?*}
sessionPhase' = *sessionPhase*
sessionId' = *sessionId*
permMode' = *permMode*
toolsThisTurn' = *toolsThisTurn*
activeSubagents' = *activeSubagents*
context' = *context*
pendingPrompt' = *pendingPrompt*
pendingResponse' = *pendingResponse*

9.2 PreToolHookAllow

PreToolHookAllow

LuxFrame

hookDecision? : *PermissionDecision*

needsPermission? : *ZBOOL*

sessionPhase = *spProcessing*
toolPhase = *tpPreHook*
hookDecision? = *pdAllow*
toolPhase' = **if** *needsPermission?* = *ztrue* **then** *tpPermission* **else** *tpExecuting*
sessionPhase' = *sessionPhase*
sessionId' = *sessionId*
permMode' = *permMode*
currentTool' = *currentTool*
toolsThisTurn' = *toolsThisTurn*
activeSubagents' = *activeSubagents*
context' = *context*
pendingPrompt' = *pendingPrompt*
pendingResponse' = *pendingResponse*

9.3 PreToolHookDeny

PreToolHookDeny

LuxFrame

hookDecision? : *PermissionDecision*

denialContext? : *CONTEXTCHUNK*

sessionPhase = *spProcessing*

toolPhase = *tpPreHook*

hookDecision? = *pdDeny*

toolPhase' = *tpNone*

currentTool' = $\emptyset$

context' = $(1 \dots \maxContext) \triangleleft (context \frown \langle denialContext? \rangle)$

sessionPhase' = *sessionPhase*

sessionId' = *sessionId*

permMode' = *permMode*

toolsThisTurn' = *toolsThisTurn*

activeSubagents' = *activeSubagents*

pendingPrompt' = *pendingPrompt*

pendingResponse' = *pendingResponse*

9.4 GrantPermission

GrantPermission

LuxFrame

sessionPhase = *spProcessing*

toolPhase = *tpPermission*

toolPhase' = *tpExecuting*

sessionPhase' = *sessionPhase*

sessionId' = *sessionId*

permMode' = *permMode*

currentTool' = *currentTool*

toolsThisTurn' = *toolsThisTurn*

activeSubagents' = *activeSubagents*

context' = *context*

pendingPrompt' = *pendingPrompt*

pendingResponse' = *pendingResponse*

9.5 DenyPermission

DenyPermission

LuxFrame

denialContext? : *CONTEXTCHUNK*

sessionPhase = *spProcessing*

toolPhase = *tpPermission*

toolPhase' = *tpNone*

currentTool' = $\emptyset$

context' = $(1 \dots \maxContext) \triangleleft (context \frown \langle denialContext? \rangle)$

sessionPhase' = *sessionPhase*

sessionId' = *sessionId*

permMode' = *permMode*

toolsThisTurn' = *toolsThisTurn*

activeSubagents' = *activeSubagents*

pendingPrompt' = *pendingPrompt*

pendingResponse' = *pendingResponse*

9.6 CompleteToolSuccess

CompleteToolSuccess

LuxFrame

output? : *CONTEXTCHUNK*

sessionPhase = *spProcessing*
toolPhase = *tpExecuting*
toolPhase' = *tpNone*
currentTool' = $\emptyset$
toolsThisTurn' = *toolsThisTurn* + 1
context' = $(1 \dots \text{maxContext}) \triangleleft (\text{context} \hat{\ } \langle \text{output?} \rangle)$
sessionPhase' = *sessionPhase*
sessionId' = *sessionId*
permMode' = *permMode*
activeSubagents' = *activeSubagents*
pendingPrompt' = *pendingPrompt*
pendingResponse' = *pendingResponse*

9.7 CompleteToolFailure

CompleteToolFailure

LuxFrame

errorContext? : *CONTEXTCHUNK*

sessionPhase = *spProcessing*
toolPhase = *tpExecuting*
toolPhase' = *tpNone*
currentTool' = $\emptyset$
toolsThisTurn' = *toolsThisTurn* + 1
context' = $(1 \dots \text{maxContext}) \triangleleft (\text{context} \hat{\ } \langle \text{errorContext?} \rangle)$
sessionPhase' = *sessionPhase*
sessionId' = *sessionId*
permMode' = *permMode*
activeSubagents' = *activeSubagents*
pendingPrompt' = *pendingPrompt*
pendingResponse' = *pendingResponse*

10 Lux Display Operations

10.1 EnableDisplay

User calls `/lux y`. Enables visual output mode.

EnableDisplay

Δ State

```
sessionPhase = spProcessing
toolPhase = tpNone
displayMode' = dmOn
sessionPhase' = sessionPhase
sessionId' = sessionId
permMode' = permMode
toolPhase' = toolPhase
currentTool' = currentTool
toolsThisTurn' = toolsThisTurn
activeSubagents' = activeSubagents
context' = context
pendingPrompt' = pendingPrompt
pendingResponse' = pendingResponse
activeScene' = activeScene
sceneKind' = sceneKind
pendingRecv' = pendingRecv
beadsBoardStale' = beadsBoardStale
```

10.2 DisableDisplay

User calls `/lux n`. Disables visual output and clears the display.

DisableDisplay

Δ State

```
sessionPhase = spProcessing
toolPhase = tpNone
displayMode' = dmOff
activeScene' =  $\emptyset$ 
sceneKind' =  $\emptyset$ 
pendingRecv' = zfalse
beadsBoardStale' = zfalse
sessionPhase' = sessionPhase
sessionId' = sessionId
permMode' = permMode
toolPhase' = toolPhase
currentTool' = currentTool
toolsThisTurn' = toolsThisTurn
activeSubagents' = activeSubagents
context' = context
pendingPrompt' = pendingPrompt
pendingResponse' = pendingResponse
```

10.3 ShowScene

Claude calls `lux show()` to display a scene. Requires display mode enabled.

ShowScene

$\Delta State$

scene? : *SCENEID*

kind? : *SceneKind*

```
sessionPhase = spProcessing
toolPhase = tpNone
displayMode = dmOn
activeScene' = {scene?}
sceneKind' = {kind?}
pendingRecv' = zfalse
beadsBoardStale' = if kind? = skBeads then zfalse else beadsBoardStale
sessionPhase' = sessionPhase
sessionId' = sessionId
permMode' = permMode
toolPhase' = toolPhase
currentTool' = currentTool
toolsThisTurn' = toolsThisTurn
activeSubagents' = activeSubagents
context' = context
pendingPrompt' = pendingPrompt
pendingResponse' = pendingResponse
displayMode' = displayMode
```

10.4 UpdateScene

Claude calls `lux update()` to patch the active scene. Requires an active scene.

UpdateScene

$\Delta State$

```
sessionPhase = spProcessing
toolPhase = tpNone
activeScene  $\neq \emptyset$ 
beadsBoardStale' = zfalse
sessionPhase' = sessionPhase
sessionId' = sessionId
permMode' = permMode
toolPhase' = toolPhase
currentTool' = currentTool
toolsThisTurn' = toolsThisTurn
activeSubagents' = activeSubagents
context' = context
pendingPrompt' = pendingPrompt
pendingResponse' = pendingResponse
displayMode' = displayMode
activeScene' = activeScene
sceneKind' = sceneKind
pendingRecv' = pendingRecv
```

10.5 ClearDisplay

Claude calls `lux clear()`.

ClearDisplay

$\Delta State$

```
sessionPhase = spProcessing
toolPhase = tpNone
activeScene' =  $\emptyset$ 
sceneKind' =  $\emptyset$ 
pendingRecv' = zfalse
beadsBoardStale' = zfalse
sessionPhase' = sessionPhase
sessionId' = sessionId
permMode' = permMode
toolPhase' = toolPhase
currentTool' = currentTool
toolsThisTurn' = toolsThisTurn
activeSubagents' = activeSubagents
context' = context
pendingPrompt' = pendingPrompt
pendingResponse' = pendingResponse
displayMode' = displayMode
```

10.6 WaitForInteraction

Claude calls `lux recv()` to block for user interaction.

WaitForInteraction

$\Delta State$

```
sessionPhase = spProcessing
toolPhase = tpNone
activeScene  $\neq \emptyset$ 
pendingRecv = zfalse
pendingRecv' = ztrue
sessionPhase' = sessionPhase
sessionId' = sessionId
permMode' = permMode
toolPhase' = toolPhase
currentTool' = currentTool
toolsThisTurn' = toolsThisTurn
activeSubagents' = activeSubagents
context' = context
pendingPrompt' = pendingPrompt
pendingResponse' = pendingResponse
displayMode' = displayMode
activeScene' = activeScene
sceneKind' = sceneKind
beadsBoardStale' = beadsBoardStale
```

10.7 ReceiveInteraction

User interacts with the display (button click, row selection, etc.).

ReceiveInteraction

$\Delta State$

interactionContext? : *CONTEXTCHUNK*

```
sessionPhase = spProcessing
pendingRecv = ztrue
pendingRecv' = zfalse
context' = (1 .. maxContext)  $\triangleleft$  (context  $\hat{\ } \langle interactionContext? \rangle$ )
sessionPhase' = sessionPhase
sessionId' = sessionId
permMode' = permMode
toolPhase' = toolPhase
currentTool' = currentTool
toolsThisTurn' = toolsThisTurn
activeSubagents' = activeSubagents
pendingPrompt' = pendingPrompt
pendingResponse' = pendingResponse
displayMode' = displayMode
activeScene' = activeScene
sceneKind' = sceneKind
beadsBoardStale' = beadsBoardStale
```

11 Consumer Integration Operations

These operations model how other tools trigger lux displays. They are gated by `displayMode = dmOn` — when lux is disabled, consumers degrade to text-only.

11.1 MarkBeadsBoardStale

A bead state change occurs (`bd update`, `bd close`, `bd create`). If the beads board is the active scene, mark it stale so Claude knows to refresh.

MarkBeadsBoardStale

$\Delta State$

```
sessionPhase = spProcessing
toolPhase = tpNone
displayMode = dmOn
sceneKind = {skBeads}
beadsBoardStale' = ztrue
sessionPhase' = sessionPhase
sessionId' = sessionId
permMode' = permMode
toolPhase' = toolPhase
currentTool' = currentTool
toolsThisTurn' = toolsThisTurn
activeSubagents' = activeSubagents
context' = context
pendingPrompt' = pendingPrompt
pendingResponse' = pendingResponse
displayMode' = displayMode
activeScene' = activeScene
sceneKind' = sceneKind
pendingRecv' = pendingRecv
```

12 Subagent Operations

SpawnSubagent

LuxFrame

agentId? : *AGENTID*

sessionPhase = *spProcessing*
agentId? $\notin$ *activeSubagents*
 $\#activeSubagents < maxSubagents$
activeSubagents' = *activeSubagents* $\cup$ {*agentId?*}
sessionPhase' = *sessionPhase*
sessionId' = *sessionId*
permMode' = *permMode*
toolPhase' = *toolPhase*
currentTool' = *currentTool*
toolsThisTurn' = *toolsThisTurn*
context' = *context*
pendingPrompt' = *pendingPrompt*
pendingResponse' = *pendingResponse*

StopSubagent

LuxFrame

agentId? : *AGENTID*

hookResult? : *HookDecision*

resultContext? : *CONTEXTCHUNK*

sessionPhase = *spProcessing*
agentId? $\in$ *activeSubagents*
hookResult? = *hdAllow*
activeSubagents' = *activeSubagents* $\setminus$ {*agentId?*}
context' = $(1 \dots maxContext) \triangleleft (context \frown \langle resultContext? \rangle)$
sessionPhase' = *sessionPhase*
sessionId' = *sessionId*
permMode' = *permMode*
toolPhase' = *toolPhase*
currentTool' = *currentTool*
toolsThisTurn' = *toolsThisTurn*
pendingPrompt' = *pendingPrompt*
pendingResponse' = *pendingResponse*

13 Response, Stop, Context, and Session End

BeginResponse

LuxFrame

response? : *RESPONSE*

sessionPhase = *spProcessing*
toolPhase = *tpNone*
activeSubagents = $\emptyset$
sessionPhase' = *spResponding*
pendingResponse' = {*response?*}
pendingPrompt' = $\emptyset$
sessionId' = *sessionId*
permMode' = *permMode*
toolPhase' = *tpNone*
currentTool' = $\emptyset$
toolsThisTurn' = *toolsThisTurn*
activeSubagents' = *activeSubagents*
context' = *context*

FinishResponse

*LuxFrame**hookResult?* : *HookDecision**responseContext?* : *CONTEXTCHUNK*

sessionPhase = *spResponding*
hookResult? = *hdAllow*
sessionPhase' = *spIdle*
pendingResponse' = $\emptyset$
context' = $(1 \dots \text{maxContext}) \triangleleft (\text{context} \hat{\ } \langle \text{responseContext?} \rangle)$
toolsThisTurn' = 0
sessionId' = *sessionId*
permMode' = *permMode*
toolPhase' = *tpNone*
currentTool' = $\emptyset$
activeSubagents' = *activeSubagents*
pendingPrompt' = $\emptyset$

ForceContinue

*LuxFrame**hookResult?* : *HookDecision*

sessionPhase = *spResponding*
hookResult? = *hdBlock*
sessionPhase' = *spProcessing*
pendingResponse' = $\emptyset$
sessionId' = *sessionId*
permMode' = *permMode*
toolPhase' = *tpNone*
currentTool' = $\emptyset$
toolsThisTurn' = *toolsThisTurn*
activeSubagents' = *activeSubagents*
context' = *context*
pendingPrompt' = *pendingPrompt*

CompactContext

*LuxFrame**newLen?* : $\mathbb{N}$

sessionPhase = *spIdle*
 $\#context > 0$
newLen? < $\#context$
newLen? $\leq \text{maxContext}$
context' = $(1 \dots \text{newLen?}) \triangleleft context$
sessionPhase' = *sessionPhase*
sessionId' = *sessionId*
permMode' = *permMode*
toolPhase' = *toolPhase*
currentTool' = *currentTool*
toolsThisTurn' = *toolsThisTurn*
activeSubagents' = *activeSubagents*
pendingPrompt' = *pendingPrompt*
pendingResponse' = *pendingResponse*

EndSession
 $\Delta State$
reason? : *SessionEndReason*

sessionPhase = *spIdle*
activeSubagents = $\emptyset$
sessionPhase' = *spEnded*
sessionId' = $\emptyset$
toolPhase' = *tpNone*
currentTool' = $\emptyset$
toolsThisTurn' = 0
activeSubagents' = $\emptyset$
context' = $\langle \rangle$
pendingPrompt' = $\emptyset$
pendingResponse' = $\emptyset$
permMode' = *permMode*
displayMode' = *dmOff*
activeScene' = $\emptyset$
sceneKind' = $\emptyset$
pendingRecv' = *zfalse*
beadsBoardStale' = *zfalse*

14 System Invariants

1. **Scene requires display:** $displayMode = dmOff \Rightarrow activeScene = \emptyset$. No scene can exist when the display is disabled.
2. **No orphan interactions:** $pendingRecv = ztrue \Rightarrow activeScene \neq \emptyset$. Cannot block for interaction without an active scene.
3. **Scene kind consistency:** $activeScene = \emptyset \Rightarrow sceneKind = \emptyset$ and $activeScene \neq \emptyset \Rightarrow \#sceneKind = 1$.
4. **Beads staleness scoping:** $beadsBoardStale = ztrue \Rightarrow sceneKind = \{skBeads\}$. Staleness only applies when the beads board is the active scene.
5. **Display mode gates consumers:** *ShowScene* and *MarkBeadsBoardStale* require $displayMode = dmOn$. When lux is off, all visual operations are refused.
6. **Clean session boundaries:** all lux state cleared on *EndSession*.

15 Consumer Integration Patterns

Lux is a building block. These patterns describe how consumers discover and use lux, following *integration.md*'s tiered protocol.

15.1 L0: Presence Detection

Consumers check for lux via MCP tool availability (L1 discovery):

```
# In consumer's hook handler:
# Check if lux MCP tools are available in this session
# If available AND displayMode = dmOn: render visual output
# If not: degrade to text-only
```

The *displayMode* flag is the L3 state that consumers read. When */lux y* is called, it writes to a config file that consumers can check.

15.2 Beads Integration

Trigger: PostToolUse on Bash commands matching `bd create|update|close`.

Pattern: When lux is enabled and the beads board is active, mark it stale. Claude sees the staleness context and re-renders the board with fresh data.

Degradation: Without lux, `bd ready` outputs text. With lux, it renders a filterable board with search, filters, and detail panel.

15.3 Quarry Integration

Trigger: PostToolUse on quarry find.

Pattern: When lux is enabled, render search results as an interactive data explorer (filterable table with document detail panel). Without lux, results are inline text.

15.4 PR Workflow Integration

Trigger: PostToolUse on `mcp__github__create_pull_request`.

Pattern: When lux is enabled, display a PR dashboard showing status, checks, review comments. Without lux, PR status is text-only in the conversation.

15.5 Z-Spec Integration

Trigger: After `/z-spec:test` or `/z-spec:check`.

Pattern: When lux is enabled, display model-check results as a dashboard (states explored, transitions fired, operations covered, invariant status). The `claude-code.lux` file we created earlier is an example of this pattern.

15.6 Session Dashboard

Trigger: SessionStart or explicit `/lux dashboard`.

Pattern: Compose a session overview from multiple sources: git status (working tree state), beads (active issues), biff (unread messages, active wall), vox (notify mode). Each data source degrades independently.

16 Hook Event Map

Hook	Matcher	Operation	Lux Effect	Blocks?
SessionStart	*	StartSession	Load display config	No
PostToolUse	lux__*	(suppress output)	Format panel	No
PostToolUse	Bash (bd *)	MarkBeadsBoardStale	Flag refresh	No
PostToolUse	quarry__find	(consumer)	Data explorer	No
PostToolUse	github__*_pr	(consumer)	PR dashboard	No
Stop	*	(consumer)	Summary visual	No
SessionEnd	*	EndSession	Clear display	No

Note: Lux itself registers only SessionStart and PostToolUse (suppress-output) hooks. The consumer integration hooks (Bash bd, quarry find, github PR) are registered by the *consuming* plugins or by session-level configuration. Lux as a building block has no upstream awareness.

17 Implementation Validation

17.1 Built and Correct

Component	Status	Notes
SessionStart hook	Correct	Deploys commands, auto-allows tools
PostToolUse suppress	Correct	Two-channel display for all MCP tools
MCP tools (show, update, clear, recv, ping)	Correct	10 tools exposed
Beads board skill	Correct	/lux:beads renders filterable board
Dashboard skill	Correct	/lux:dashboard composes metrics+charts
Data explorer skill	Correct	/lux:data-explorer with built-in filters
Diagram skill	Correct	/lux:diagram with auto-layout

17.2 Not Yet Built

Feature	Z Operation	Effort	Notes
/lux y and /lux n toggle	EnableDisplay/DisableDisplay	Medium	Needs config file + command
Display mode config	(L3 state)	Low	.lux/config.md with displayMode
Beads board auto-refresh	MarkBeadsBoardStale	Medium	PostToolUse on Bash(bd *)
Quarry search display	(consumer pattern)	Low	PostToolUse on quarry find
PR dashboard	(consumer pattern)	Medium	PostToolUse on github PR
Z-spec results display	(consumer pattern)	Low	Already have .lux file pattern
Session dashboard	(consumer pattern)	Medium	Compose from multiple sources

17.3 Design Note: Building Block Boundary

Per `integration.md`, lux is a building block with no upstream awareness. The consumer integration operations in this Z spec model what *consumers do* with lux, not what lux does itself. In implementation:

- `ShowScene`, `UpdateScene`, `ClearDisplay`, `WaitForInteraction`, `ReceiveInteraction` are lux MCP tools — implemented in lux.
- `EnableDisplay`, `DisableDisplay` are lux commands — implemented in lux.
- `MarkBeadsBoardStale` would be a `PostToolUse` hook in the consuming plugin (biff or a shared integration hook), not in lux.
- Consumer display triggers (quarry search, PR dashboard) are implemented in the consumer's hooks or skills, not in lux.

This respects the one-way arrow: consumers discover lux, lux never discovers consumers.